FOUNDERS OF AgriScan-AI

- Andzani Singh Mavangwa
- Nhlayiseko Pride Mathebula
- Risima Marvin Kubayi
- Mikhenso Trust Mkhonto



AgriScanAI

Detect • Protect • Grow

BUSINESS BACKGROUND

The Problem:

- Global food demand increasing, sustainable farming critical
- Shortage of plant disease experts in South Africa and developing countries
- Small farmers in rural areas lack access to plant doctors
- Resource constraints prevent timely disease detection

Impact:

- Up to 40% crop losses annually due to pests and diseases
- Global economic cost: >$220 billion annually

Our Response:

- AI tool using leaf image analysis for disease detection
- Reduces workload for agricultural experts
- Provides fast, accurate predictions with confidence levels
- Optimizes pesticide use for environmental benefits

The Problem

. Plant diseases can spread quickly, reducing crop yield and quality.

• In South Africa and other developing regions, farmers may have limited access to plant pathologists and agricultural advice.

• Manual leaf inspection takes time and can lead to missed symptoms or incorrect diagnosis.

• Early identification helps farmers respond sooner, protect harvests and support food security.

Our Solution

- AgriScan-AI is an AI-powered decision-support tool that analyses plant leaf images and predicts likely disease classes.
- Uses the PlantVillage dataset, with about 54,000+ images across 38 classes.
- Applies supervised learning and a Convolutional Neural Network (CNN).
- Returns a predicted class, confidence score and top-three predictions.
- Provides disease information, symptoms, prevention guidance and recommended actions

Our Objectives

- Detect and classify plant foliar diseases from leaf images.
- Aim for accurate, timely results using the PlantVillage dataset.
- Help farmers and agricultural officers make informed crop-management decisions.
- Reduce avoidable crop losses and support efficient pesticide use.
- Make AI-supported guidance more accessible in remote and low-resource farming areas.

Requirements

FUNCTIONAL REQUIREMENTS

- Accept leaf images in JPG and PNG formats.
- Resize and preprocess images before analysis.
- Classify images into 38 disease classes and return a confidence score.
- Show the top 3 predictions and probabilities.
- Generate confusion matrices and classification reports.
- Provide disease symptoms, prevention and recommended actions.

.

NON-FUNCTIONAL REQUIREMENTS

- Process at least 30 leaf images per hour concurrently.
- Maintain consistent performance across image conditions.
- Support future crop and disease expansion.
- Target 99% availability during the growing season.
- Provide a CLI and API for integration.

Success Criteria

- Target more than 85% accuracy across 38 disease classes.
- Process 1,000 test images and generate evaluation reports in under 5 minutes on a computer with less than 16 GB RAM.
- Classify a single leaf image and return a confidence score in under 3 seconds.
- Target adoption by more than 70% of participating farmers and agricultural officers.
- Provide practical disease information and guidance with predictions.

Risks & Mitigation

Risk	Mitigation
Misclassification	Normalize images; show confidence
Over-reliance	Decision-support disclaimer
Poor image quality	Photo guidance and input checks
Dataset bias	Class weights and augmentation
Technical failure	Backups and version control

Constraints

- The current system analyses leaf images only; roots, stems, fruits and other farm tasks are out of scope.
- Classification is limited to the 38 classes in the PlantVillage dataset.
- Validation is limited to data similar to PlantVillage.
- Testing is planned on limited computing resources (documentation specifies less than 8 GB RAM).
- Clear, bright leaf photographs work best; low-quality images may reduce reliability.
- The interface and disease information are in English.

Tools and Techniques

PROGRAMMING & LIBRARIES

- • Python
- • TensorFlow and Keras
- • NumPy and scikit-learn
- • PIL / Pillow
- • Matplotlib
- • JSON, argparse and os

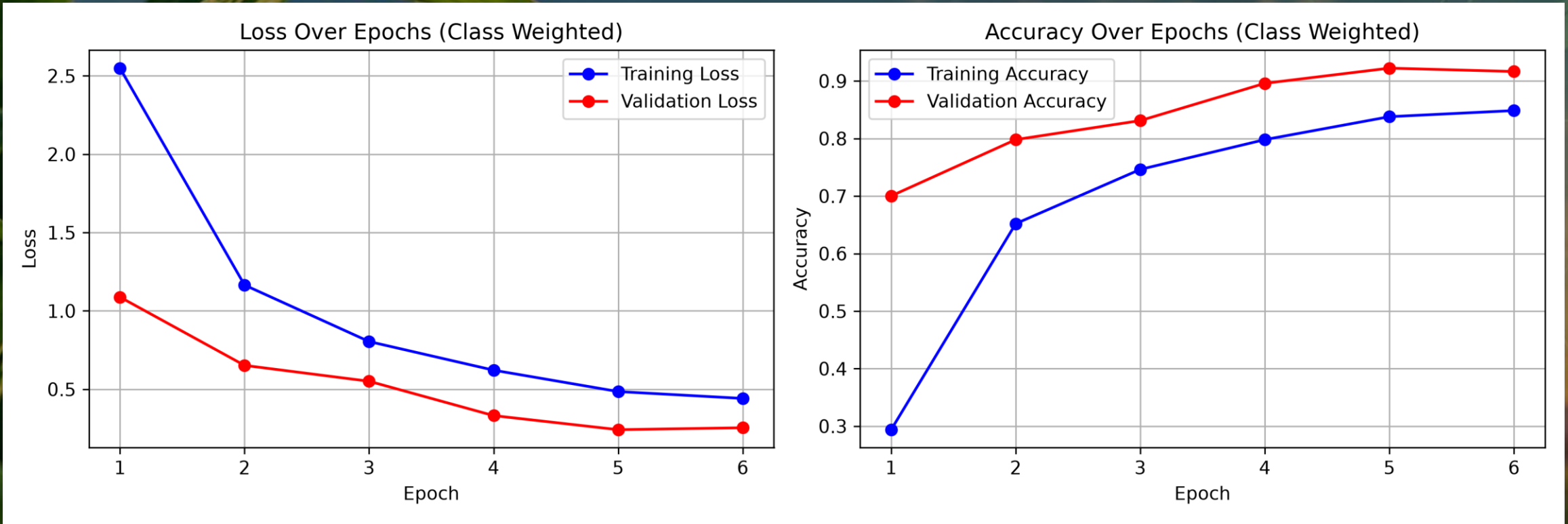
TECHNIQUES

- • Supervised image classification
- • Convolutional Neural Networks
- • Image normalization and augmentation
- • Class weighting, dropout and early stopping

COLLABORATION

- • Git / GitHub
- • Visual Studio Code
- • WhatsApp, Word and Canva

Accuracy and Loss Curves Class Weighted



Loss Over Epoch:

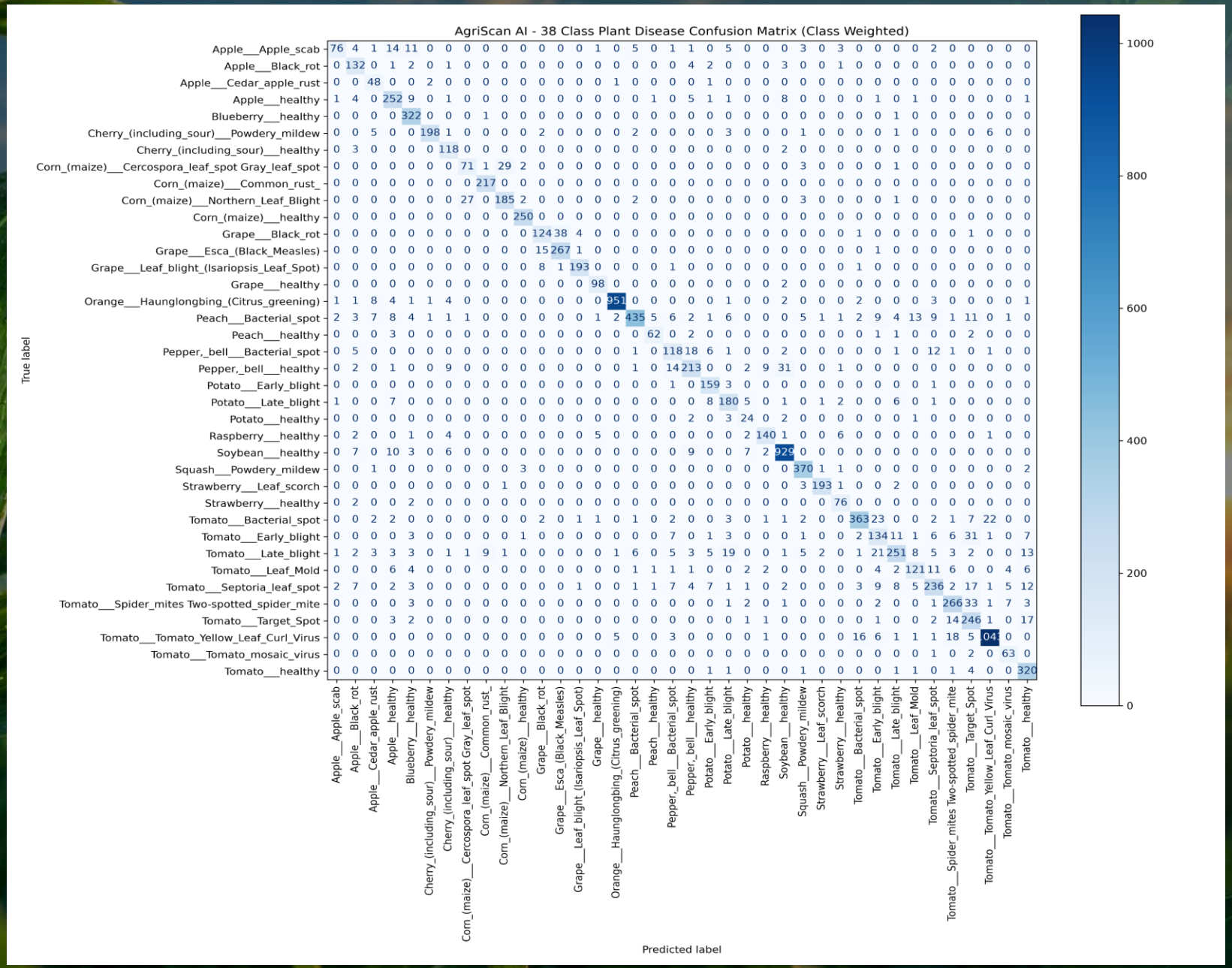
- Training loss measures error on training data
- Validation loss measures error on unseen data

Accuracy Over Epoch:

- Training accuracy shows correct predictions on training data
- validation accuracy on unseen data

EVALUATION METRICS GRAPH

Confusion Matrix:



Our Poster

AgriScanAI

Plant Foliar Disease Diagnostic System

Artificial Intelligence for Sustainable Agriculture.

AgriScan AI uses Artificial Intelligence (AI) to identify plant diseases from leaf images. It helps farmers detect diseases faster and make better farming decisions.

PROBLEMS

- Manual diagnosis is slow & expensive
- Excessive pesticide use harm the environment.
- Farmers lack access to experts
- Late diagnosis can reduce crop yields.

SOLUTIONS

- CNN-based automated disease detection
- Real-time prediction from leaf images
- 38 disease types across 13 crops
- Instant results with confidence scores.

PREDICTION



Plant: Corn(Maize)

Disease: Healthy Corn Leaf

Status: HEALTHY

Confidence: 100.00%

HOW IT WORKS

- **Input:** Plant leaf image.
- CNN model analyzes patterns.
- **Output:** Disease class + confidence.
- Provides symptoms, prevention, treatment.

RESULTS

- Accuracy: ≥85%
- Prediction time: <3 seconds.
- Confident & reliable diagnoses.

DATA

- Plant Village dataset: 54,000+ images.
- 38 disease classes.
- 13 crop types.
- Preprocessed & augmented.

KEY FEATURES

- Instant diagnosis.
- Top 3 predictions.
- Disease knowledge base.
- CLI & API access.

MODEL

- 4-layer CNN architecture.
- Input: 128x128 RGB images.
- ~2.5M parameters.
- Trained with class weighting.

IMPACT

- Reduces costs & increases yields.
- Democratizes expert knowledge.
- Reduces pesticide use.
- Sustainable farming.

Global Imports – AgriScan-AI

```
+"""
AgriScan AI - Plant Foliar Disease Diagnostic System
Train: python AgriScanAI.py --train-class-weighted
Predict: python AgriScanAI.py --predict "path/to/image.jpg"
"""
import os
import json
import argparse
import random
import numpy as np
import tensorflow as tf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from keras.models import Sequential
from keras.utils import image_dataset_from_directory
from keras import layers, metrics
from keras.optimizers import Adam
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay
from sklearn.utils.class_weight import compute_class_weight
from disease_knowledge_base import get_disease_information

SEED = 2025
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

AUTOTUNE = tf.data.AUTOTUNE
IMAGE_SIZE = (128, 128)
BATCH_SIZE = 64

```

Dataset Load Function

```python
#Function to load the dataset
def load_datasets():
    """Load and preprocess train/val/test datasets. Returns datasets and class names."""
    base_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset")
    train_dir, test_dir = os.path.join(base_dir, "train"), os.path.join(base_dir, "test")

    for d in [base_dir, train_dir, test_dir]:
        if not os.path.exists(d):
            raise FileNotFoundError(f"Directory not found: {d}")

    class_names = sorted([f for f in os.listdir(train_dir) if os.path.isdir(os.path.join(train_dir, f))])
    if len(class_names) == 0 or len(class_names) != len(set(class_names)):
        raise ValueError("Invalid or duplicate class names in training directory")

    print(f"Found {len(class_names)} disease classes")

    train_ds = image_dataset_from_directory(train_dir, labels="inferred", label_mode="categorical",
        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE,
        shuffle=True, seed=SEED, validation_split=0.2, subset="training")

    val_ds = image_dataset_from_directory(train_dir, labels="inferred", label_mode="categorical",
        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE,
        shuffle=False, seed=SEED, validation_split=0.2, subset="validation")

    test_ds = image_dataset_from_directory(test_dir, labels="inferred", label_mode="categorical",
        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE, shuffle=False)

    test_class_names = sorted([f for f in os.listdir(test_dir) if os.path.isdir(os.path.join(test_dir, f))])
    if set(test_class_names) != set(class_names):
        raise ValueError(f"Class mismatch between train and test directories")

    rescale = layers.Rescaling(1.0 / 255.0)
    augment = Sequential([layers.RandomFlip("horizontal"), layers.RandomZoom(0.1), layers.RandomContrast(0.1)], name="augment")

    train_ds = train_ds.map(lambda x, y: (rescale(augment(x, training=True)), y), num_parallel_calls=AUTOTUNE).cache().prefetch(AUTOTUNE)
    val_ds = val_ds.map(lambda x, y: (rescale(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
    test_ds = test_ds.map(lambda x, y: (rescale(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)

```

Build CNN Function

```python
#Function to build the CNN model
def build_model(class_names):
    """Build and compile CNN model."""
    model = Sequential(name="agriscan_cnn")
    model.add(layers.Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3)))

    filters = 32
    for _ in range(3):
        model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
        model.add(MaxPooling2D(pool_size=(2, 2)))
        filters *= 2
    model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Flatten())
    model.add(Dense(256, activation="relu"))
    model.add(Dropout(0.5))
    model.add(Dense(len(class_names), activation="softmax"))

    model.compile(optimizer=Adam(learning_rate=0.001), loss="categorical_crossentropy",
        metrics=["accuracy", metrics.Precision(name="precision"), metrics.Recall(name="recall")])
    model.summary()
    return model
```

Evaluate Model Function

```python
#Function to evaluate the model
def evaluate_model(model, test_ds, class_names, title_suffix, output_suffix):
    """Evaluate model, generate confusion matrix and classification report."""
    loss, acc, prec, rec = model.evaluate(test_ds, verbose=1)[:4]
    print(f"Test - Loss: {loss:.4f}, Acc: {acc:.4f}, Prec: {prec:.4f}, Rec: {rec:.4f}")

    predictions = model.predict(test_ds, verbose=1)
    if not np.all(np.isfinite(predictions)):
        raise ValueError("Model predictions contain non-finite values (NaN or Inf)")

    y_pred = np.argmax(predictions, axis=1)
    y_true = np.concatenate([np.argmax(labels.numpy(), axis=1) for _, labels in test_ds])

    cm = confusion_matrix(y_true, y_pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=class_names)
    fig, ax = plt.subplots(figsize=(16, 14))
    disp.plot(ax=ax, cmap="Blues", values_format="d", xticks_rotation=90)
    plt.title(f"AgriScan AI - 38 Class Plant Disease Confusion Matrix {title_suffix}")
    plt.tight_layout()
    plt.savefig(f"results/confusion_matrix{output_suffix}.png", dpi=300)
    plt.close(fig)
    print("Confusion matrix saved")

    report = classification_report(y_true, y_pred, target_names=class_names, digits=4, zero_division=0)
    print(f"\nClassification Report {title_suffix}:")
    print(report)

    report_dict = classification_report(y_true, y_pred, target_names=class_names, digits=4, output_dict=True, zero_division=0)
    with open(f"results/classification_report{output_suffix}.json", "w", encoding="utf-8") as f:
        json.dump(report_dict, f, indent=4)
    print("Classification report saved")
```

Calculate Class Weights Function

```python
#Function to calculate class weights
def calculate_class_weights(train_ds, class_names):
    """Calculate balanced class weights from training dataset labels."""
    print("Calculating class weights from training data...")
    y_train = np.concatenate([np.argmax(labels.numpy(), axis=1) for _, labels in train_ds])

    class_weights = compute_class_weight(class_weight='balanced', classes=np.arange(len(class_names)), y=y_train)
    class_weight_dict = dict(enumerate(class_weights))

    print(f"Class Weight Statistics: Min: {min(class_weights):.4f}, Max: {max(class_weights):.4f}, Mean: {np.mean(class_weights):.4f}")

    for class_name in ['Apple___Apple_scab', 'Apple___Cedar_apple_rust', 'Potato___healthy', 'Raspberry___healthy', 'Orange___Haunglongbing_(Citrus_greening)']:
        if class_name in class_names:
            print(f"  {class_name}: {class_weights[class_names.index(class_name)]:.4f}")

    return class_weight_dict

```

Train CNN Function

```python
#Function to train model with class weights
def train_model_with_class_weights(epochs=6):
    """Train model with class weights to handle imbalance."""
    if not isinstance(epochs, int) or epochs <= 0:
        raise ValueError(f"Epochs must be a positive integer, got: {epochs}")

    os.makedirs("models", exist_ok=True)
    os.makedirs("results", exist_ok=True)

    train_ds, val_ds, test_ds, class_names = load_datasets()
    model = build_model(class_names)
    class_weight_dict = calculate_class_weights(train_ds, class_names)

    with open("models/class_names_class_weighted.json", "w", encoding="utf-8") as f:
        json.dump(class_names, f, indent=4)

    with open("results/class_weights.json", "w", encoding="utf-8") as f:
        json.dump({str(k): float(v) for k, v in class_weight_dict.items()}, f, indent=4)

    callbacks = [
        EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True, verbose=1),
        ModelCheckpoint(filepath="models/class_weighted_model.keras", monitor="val_accuracy", save_best_only=True, verbose=1),
        ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-5, verbose=1)
    ]

    print(f"\nTraining with class weights for {epochs} epochs...")
    history = model.fit(train_ds, validation_data=val_ds, epochs=epochs, class_weight=class_weight_dict, callbacks=callbacks, verbose=1)

```

Load Trained Model Function

```python
#Function to load the trained model
def load_model():
    """Load class-weighted model and class names with validation."""
    models_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "models")
    model_path = os.path.join(models_dir, "class_weighted_model.keras")
    class_path = os.path.join(models_dir, "class_names_class_weighted.json")

    if not os.path.exists(model_path):
        raise FileNotFoundError(f"Class-weighted model not found: {model_path}\nRun training first with: python AgriScanAI.py --train-class-weighted")
    if not os.path.exists(class_path):
        raise FileNotFoundError(f"Class-weighted class names not found: {class_path}\nRun training first with: python AgriScanAI.py --train-class-weighted")

    print("Loading class-weighted model (improved performance for minority classes).")

    with open(class_path, "r", encoding="utf-8") as f:
        class_names = json.load(f)

    if not isinstance(class_names, list) or len(class_names) == 0 or len(class_names) != len(set(class_names)):
        raise ValueError("Invalid or duplicate class names in JSON")

    model = tf.keras.models.load_model(model_path)
    if model.output_shape[-1] != len(class_names):
        raise ValueError(f"Model output dimension ({model.output_shape[-1]}) does not match number of classes ({len(class_names)})")

    return model, class_names

```

Predict on single image Function

```python
#Function to predict disease from image
def predict_image(image_input, model=None, class_names=None):
    """Predict disease from image (path, file object, PIL Image, or numpy array)."""
    if model is None or class_names is None:
        model, class_names = load_model()

    resolved_path = None
    img_array = None

    if isinstance(image_input, str):
        resolved_path = validate_path(image_input)
        if not resolved_path:
            raise FileNotFoundError(f"Image not found: {image_input}")
        img = tf.keras.utils.load_img(resolved_path, target_size=IMAGE_SIZE, color_mode="rgb", interpolation="bilinear")
        img_array = tf.keras.utils.img_to_array(img, dtype="float32")
    elif hasattr(image_input, "read"):
        pil_img = Image.open(image_input)
        if pil_img.mode not in ["RGB", "RGBA", "L", "P"]:
            raise ValueError(f"Unsupported image mode: {pil_img.mode}")
        if pil_img.mode != "RGB":
            pil_img = pil_img.convert("RGB")
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32)
    elif isinstance(image_input, Image.Image):
        if image_input.mode not in ["RGB", "RGBA", "L", "P"]:
            raise ValueError(f"Unsupported image mode: {image_input.mode}")
        pil_img = image_input.convert("RGB") if image_input.mode != "RGB" else image_input
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32)
    elif isinstance(image_input, np.ndarray):
        img_array = image_input.astype("float32")
    else:
        raise TypeError(f"Unsupported input type: {type(image_input).__name__}")

    if img_array is None or len(img_array.shape) != 3 or img_array.shape[2] not in [1, 3, 4]:
        raise ValueError(f"Invalid image array shape: {img_array.shape if img_array is not None else 'None'}")

    if img_array.shape[2] == 1:
        img_array = np.repeat(img_array, 3, axis=2)
    elif img_array.shape[2] == 4:
        img_array = img_array[:, :, :3]

    if img_array.max() > 1.0:
        img_array = img_array / 255.0

```

Validate file path Function

```
231  #Function to validate file path
232  def validate_path(path):
233      """Validate and resolve file path. Returns absolute path or None."""
234      if not path or not isinstance(path, str):
235          return None
236      path = path.strip().strip('"\'')
237      if not path:
238          return None
239      path = os.path.expanduser(os.path.normpath(path))
240      if os.path.isfile(path):
241          return os.path.abspath(path)
242      project_root = os.path.dirname(os.path.abspath(__file__))
243      rel_path = os.path.join(project_root, path)
244      if os.path.isfile(rel_path):
245          return os.path.abspath(rel_path)
246      return None
```

Main Function

```python
#Main function
def main():
    parser = argparse.ArgumentParser(description="AgriScan AI - Plant Foliar Disease Detection System")
    parser.add_argument("--train-class-weighted", action="store_true", help="Train model with class weights")
    parser.add_argument("--predict", nargs="?", const="__INTERACTIVE__", default=None, metavar="IMAGE_PATH", help="Predict disease from image")
    parser.add_argument("--epochs", type=int, default=6, help="Training epochs (default: 6)")
    args = parser.parse_args()

    if args.train_class_weighted:
        try:
            train_model_with_class_weights(args.epochs)
            print("\nClass-weighted training completed successfully!")
        except Exception as e:
            print(f"\nClass-weighted training failed: {e}")
            return 1
    elif args.predict is not None:
        predict_cli() if args.predict == "__INTERACTIVE__" else predict_cli(args.predict)
    else:
        parser.print_help()

    return 0


if __name__ == "__main__":
    exit(main())

```